String matching solutions

32-E01 Overlap, empty patterns, and comparisons

Shifts 0,1,2 are valid. Each `aaa` window takes three comparisons, for nine in total. Sharing text positions is allowed. For the empty pattern, all six boundaries 0 through 5 match. A length-6 pattern has no window in a length-5 text, so it gives no shifts. The respective outputs are `[0,1,2]`, `[0,1,2,3,4,5]`, and `[]`. Branch on these lengths before any access to `P[1]` or an initial length-m window.

32-E02 A collision is not a match

Here $h = 10, p = 15 \bmod 11 = 4$. Shift 0 has window 15, hash 4, and succeeds. The next hash is $(10(4 - 1 \cdot 10) + 2) \bmod 11 = -58 \bmod 11 = 8$, for window 52. Next, $(10(8 - 5 \cdot 10) + 6) \bmod 11 = -414 \bmod 11 = 4$, for 26. Hashes agree, but 15≠26, so reject this spurious hit.

The next update is $(10(4 - 2 \cdot 10) + 1) \bmod 11 = -159 \bmod 11 = 6$, for 61. The final update is $(10(6 - 6 \cdot 10) + 5) \bmod 11 = -535 \bmod 11 = 4$, for 15, which verifies. The hash sequence is `[4,8,4,6,4]`, and the correct shifts are `[0,4]`. Omitting verification falsely accepts shift 2. Directly reducing each window gives the same numbers and independently checks the rolling calculation.

32-E03 All borders and prefix construction

The prefix values are `[0,0,1,0,1,2,3,2,3,4]`. At $i = 8$, the character is `b` and initial $k = 3$. Since $P[4] = c$ differs, fall to $\pi[3] = 1$. Now $P[2] = b$, giving $\pi[8] = 2$. Other positions extend according to the same equality test.

The longest proper border of `abacababac` is `abac`, length 4. Since $\pi[4] = 0$, all proper border lengths are 4 and 0. Any border of positive length below 4 must also be a border of the suffix `abac`, because both prefix and suffix of that length coincide. This contradicts $\pi[4] = 0$. Any length above 4 but below 10 contradicts maximality. Repeating this argument at each failure link proves the general chain's completeness.

The only nonempty proper border is `abac`. Setting $\pi[q] = q$ prevents fallback from decreasing and can make the mismatch while loop run forever. It also violates the definition of proper border.

32-E04 Accepting states also transition

With columns `a,b`, rows 0,1,2,3 are `(1,0)`, `(1,2)`, `(3,0)`, `(1,2)`. In the last row, `abaa` has longest pattern-prefix suffix `a`, and `abab` has `ab`.

String matching solutions (continued)

The automaton scans `abababa` through states 1,2,3,2,3,2,3 and outputs shifts 0,2,4. The prefix function of `aba` is `[0,0,1]`. KMP also has state 3 at each acceptance check but retains state 1 after output. The next `b` extends that state to 2. Thus the states agree at the per-character acceptance check, while KMP's saved state can differ immediately after acceptance.

Resetting to zero leaves state zero on the next `b`, then only reaches 1 on `a`, losing shift 2. Correct fallback retains the final `a`, which is the next match's first character.

32-E05 Many comparisons for one character

The first k pattern characters are `a`, with prefix values $0, 1, \ldots, k-1$. The final `b` has prefix value zero. The first k text as raise q from 0 to k. The next `a` differs from $P[k+1] = b$, so fallback to $\pi[k] = k-1$, then match $P[k] = a$, returning to k. Every remaining `a` does the same one-step fallback and extension.

The final `c` causes $k \to k-1 \to \cdots \to 1 \to 0$, and all comparisons with `a` or `b` fail. It retains zero and performs k while-body iterations for this single character.

Across the sequence, q increases at most once per character and stays nonnegative. Each fallback lowers it by at least one, so total fallbacks cannot exceed total increases, at most n. One character may cost $\Theta(k)$, while the entire scan is still $O(n)$.

32-E06 Conditions for a random base

Let $D(z) = \sum_j (X[j] - Y[j]) z^{m-j}$. At least one coefficient is nonzero in $\mathbb{F}_q$, so this is a nonzero polynomial of degree at most $m-1$. The factor theorem and induction on degree give at most $m-1$ roots. Uniform sampling among q field elements therefore gives probability at most $(m-1)/q$, capped at 1.

If codes 0 and 5 are allowed with $q = 5$, the distinct length-1 strings `[0]` and `[5]` have coefficient difference -5, which is zero in the field. Their difference polynomial is identically zero, and every base collides. Claiming a bound of zero would be false. Injectivity of character encoding in the field is the missing premise.

For each nonmatching window let I_s indicate a collision. Linearity gives $E[\sum_s I_s] = \sum_s E[I_s]$ without independence. Inputs must be fixed before choosing the base. Selecting new strings after observing the base does not satisfy the fixed-input probability guarantee.

String matching solutions (continued)

32-E07 Streaming across chunks

Precompute $\pi = [0, 1, 0, 1, 2, 2, 3]$. Retain q and a global count i. After the first chunk, $i = 5, q = 5$. The next `a` gives $q = 6$. Then `b` gives $q = 7, i = 7$, output $i - m = 0$, and retain 3. Reading `a,a,a,b` gives states 4,5,6,7. At $i = 11$, output 4 and retain 3.

Every new-character comparison uses only the pattern, its prefix function, and q. All useful equality information from old text is represented by the longest extendible border. The proof is the ordinary KMP proof and does not depend on chunk boundaries. Accumulating global i makes outputs shifts in the original text.

The matches are `[0,4]`. Storage is $\Theta(m)$ for the pattern and prefix table and a constant number of indices. Returning original text passages or surrounding context would require an additional buffer under a different output specification.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

String matching solutions (continued)

Verification code appendix: prefix_function.py

```
"""Build the KMP prefix function and scan for overlapping matches."""

def prefix_function(pattern):
    pi = [0] * len(pattern)
    for i in range(1, len(pattern)):
        j = pi[i - 1]
        while j and pattern[i] != pattern[j]:
            j = pi[j - 1]
        if pattern[i] == pattern[j]:
            j += 1
        pi[i] = j
    return pi


def matches(text, pattern):
    if not pattern:
        return list(range(len(text) + 1))
    if len(pattern) > len(text):
        return []
    pi = prefix_function(pattern)
    result, j = [], 0
    for i, char in enumerate(text):
        while j and char != pattern[j]:
            j = pi[j - 1]
        if char == pattern[j]:
            j += 1
        if j == len(pattern):
            result.append(i - len(pattern) + 1)
            j = pi[j - 1]
    return result


assert prefix_function("abab") == [0, 0, 1, 2]
assert matches("aaaa", "aa") == [0, 1, 2]
assert matches("", "") == [0]
assert matches("abc", "") == [0, 1, 2, 3]
assert matches("ab", "abcd") == []
print(matches("abababaca", "aba"))
```

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

String matching solutions (continued)

Verification code appendix: string_match_lab.py

```
"""Reference executable models for the chapter; indices returned are zero-based."""
def naive(text,pattern):
    return [s for s in range(len(text)-len(pattern)+1) if text[s:s+len(pattern)]==pattern]

def prefix(pattern):
    pi=[0]*len(pattern); k=0
    for i in range(1,len(pattern)):
        while k and pattern[k]!=pattern[i]:k=pi[k-1]
        if pattern[k]==pattern[i]:k+=1
        pi[i]=k
    return pi

def kmp(text,pattern):
    if not pattern:return list(range(len(text)+1))
    pi=prefix(pattern);q=0;found=[];trace=[]
    for i,a in enumerate(text):
        chain=[q]
        while q and pattern[q]!=a:q=pi[q-1];chain.append(q)
        if pattern[q]==a:q+=1
        accepted=q==len(pattern)
        if accepted:found.append(i+1-len(pattern))
        trace.append((i+1,a,chain,q,accepted))
        if accepted:q=pi[q-1]
    return found,trace

def automaton(pattern,alphabet):
    if not pattern:return {(0,a):0 for a in alphabet}
```

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

String matching solutions (continued)

```
    pi=prefix(pattern);m=len(pattern);delta={}
    for a in alphabet:delta[0,a]=int(a==pattern[0])
    for q in range(1,m+1):
        for a in alphabet:
            delta[q,a]=q+1 if q<m and a==pattern[q] else delta[pi[q-1],a]
    return delta

def rabin_karp(digits,pattern,base=10,modulus=11):
    n=len(digits);m=len(pattern)
    if m==0:return list(range(n+1)),[]
    if m>n:return [],[]
    h=pow(base,m-1,modulus);p=t=0
    for a,b in zip(pattern,digits):p=(base*p+a)%modulus;t=(base*t+b)%modulus
    found=[];trace=[]
    for s in range(n-m+1):
        same=digits[s:s+m]==pattern
        trace.append((s,t,t==p,same))
        if t==p and same:found.append(s)
        if s<n-m:t=(base*(t-digits[s]*h)+digits[s+m])%modulus
    return found,trace

if __name__=='__main__':
    print('prefix',prefix('aabaaab'))
    print('KMP',kmp('aabaaabaaab','aabaaab'))
    print('RK',rabin_karp(list(map(int,'122312')),[1,2]))
```

Lin Cing https://github.com/xinhi6716/computer-science-self-study

References and attribution

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. Introduction to Algorithms, 3rd Edition.

Original educational synthesis by Lin Cing. Original material is licensed under CC BY 4.0. The referenced textbook is not distributed with these notes.

https://github.com/xinhi6716/computer-science-self-study